11 – University Viva Questions

Below is a curated list of **≥60 viva-style questions** that a professor (or any examiner) might ask about the EasyShop project. They are grouped by difficulty and topic. For each question we provide:

- **Short answer** – a one-sentence punch-line you can say immediately.
- **Detailed answer** – a paragraph you can expand on if the examiner probes deeper (includes file/table references).
- **Reference** – the exact file or DB object you should point to for evidence.
- **Follow-up probe** – a possible next question the examiner could ask, helping you anticipate the flow.

A. Basic – "What is this project?"

#	Question	Short Answer	Detailed Answer	Reference	Follow-up Probe
A1	What is the purpose of EasyShop?	A single-vendor e-commerce web application for the Pakistani market.	It demonstrates a complete online shop: product catalog, authentication, cart, checkout, order management, admin dashboard, and security hardening – built with vanilla HTML, CSS, Bootstrap 5, JavaScript, Core PHP, and MySQLi.	`README.md` (project root)	How does it differ from a multi-vendor marketplace?
A2	Which technologies were used?	HTML5, CSS3, Bootstrap 5, vanilla JS, Core PHP 8, MySQLi, Git.	The front-end uses Bootstrap for responsiveness; PHP handles server-side rendering and business logic; MySQL stores relational data with foreign-key constraints; Git tracks the source.	`composer.json` (empty, only for PHPUnit)	Why not use a modern framework like Laravel?
A3	How is the code organised?	By functional areas – `auth/`, `admin/`, `includes/`, `actions/`, `tests/`, `database/`.	Each top-level folder contains related scripts: `auth/` for login/registration UI, `admin/` for back-office pages, `includes/` for reusable functions and config, `actions/` for POST handlers, `tests/` for PHPUnit suites, `database/` for schema files.	`documentation/learning-guide/02-Architecture-and-Folder-Structure.md`	What would you change in this structure for scalability?

B. Intermediate – Architecture & Data Flow

#	Question	Short Answer	Detailed Answer	Reference	Follow-up Probe
B1	Describe the checkout flow from cart to order creation.	The cart is read, each product row is `SELECT … FOR UPDATE`, stock is verified, an order row + order_items are inserted, stock is deducted, and the transaction commits.	In `actions/place-order.php` (lines 87-306) the script begins a MySQL transaction, locks each product row, calculates subtotals using the live price, creates a unique order number, inserts into `orders` and `order_items`, updates product stock, empties the cart, and finally commits. If any step fails, it rolls back, preserving ACID.	`actions/place-order.php` lines 87-306	Why is the `FOR UPDATE` lock necessary?
B2	How does the system prevent over-selling?	By locking product rows inside a transaction and verifying stock after the lock.	The `SELECT … FOR UPDATE` ensures that concurrent checkouts cannot read the same stock value simultaneously. After acquiring the lock, the script checks `stock >= requested_qty`; if not, it aborts and rolls back. This guarantees serialisable execution for the critical section.	`actions/place-order.php` lines 118-124	What isolation level is MySQL using by default?
B3	Explain how orders are linked to their items and why snapshots are stored.	`order_items` stores a copy of `product_id`, `price`, and `name` at purchase time.	When an order is placed, each line item is inserted with the current product `price` and `name`. This decouples historic order data from later product updates (price changes, name edits) so the order receipt remains accurate.	`database/easyshop_production.sql` – tables `orders` and `order_items`	Could you retrieve the original product data later?
B4	How does the admin dashboard compute total sales?	By aggregating `order_items.subtotal` for delivered orders.	In `admin/index.php` the query `SELECT IFNULL(SUM(oi.subtotal),0) AS total_sales FROM orders o JOIN order_items oi ON o.id = oi.order_id WHERE o.order_status = 'delivered'` returns the sum of all completed sales, displayed in a Bootstrap card.	`admin/index.php` (lines 64-79)	Why only `delivered` orders?

C. Advanced – Security & Concurrency

#	Question	Short Answer	Detailed Answer	Reference	Follow-up Probe
C1	Which OWASP Top-10 mitigations are implemented?	SQLi, XSS, CSRF, Session hijacking, IDOR, insecure direct object references, insecure file upload, and limited business-logic validation.	Detailed breakdown is in `06-Security-Explained.md` – prepared statements for SQLi, `e()` for XSS, CSRF tokens in `includes/csrf.php`, secure session cookies in `includes/functions.php`, admin/auth guards, `secure_upload_image` for file uploads, and transaction-based stock checks.	`06-Security-Explained.md`	Which mitigation is still missing?
C2	How is CSRF protection implemented?	A per-session random token stored in `$_SESSION['csrf_token']` and validated on every POST via `require_csrf()`.	`includes/csrf.php` generates a 64-character hex token, outputs a hidden field with `csrf_field()`, and `require_csrf()` aborts with 403 if the token does not match using `hash_equals`. All state-changing forms (login, register, add-to-cart, place-order, admin updates) call `require_csrf()`.	`includes/csrf.php` lines 17-66	What would happen if the token were leaked via XSS?
C3	Describe the **stock restoration** logic when an admin cancels an order.	It checks `is_stock_restored` flag, restores each product's stock, and sets the flag to `1` within a transaction.	In `admin/orders/update-status.php` (lines 84-119) the script begins a transaction, locks the order row, validates allowed statuses, loops through each `order_items` row, adds back the `quantity` to the product stock **only if** `is_stock_restored = 0`. The flag prevents double-restoration.	`admin/orders/update-status.php` lines 84-119	How would you undo a cancellation?
C4	What session settings are used to harden cookies?	`Secure` (HTTPS only), `HttpOnly`, `SameSite=Lax`.	`init_session()` in `includes/functions.php` (lines 10-30) calls `session_set_cookie_params` with those flags before `session_start()`. This protects against XSS-based cookie theft and some CSRF via third-party links.	`includes/functions.php` lines 10-30	Why not use `SameSite=Strict`?
C5	Explain the purpose of `is_stock_restored` column.	Prevents the same cancelled order from restoring stock multiple times.	It defaults to `0`. When an admin cancels, the code checks this flag; if `0`, it restores stock and flips it to `1`. Subsequent cancellation attempts see `is_stock_restored = 1` and skip restoration, avoiding inventory inflation.	`database/easyshop_production.sql` – `orders` table definition	Could this flag be removed with a different design?

D. Expert – Design Decisions & Future Work

#	Question	Short Answer	Detailed Answer	Reference	Follow-up Probe
D1	Why was a **flat shipping fee** chosen instead of carrier-based calculation?	Simplicity for the target market and limited hosting resources.	Phase 6 introduced a single `calculate_shipping()` function returning a constant `200`. This avoids external API calls, keeps the checkout fast, and matches typical local delivery models in Pakistan where sellers set a fixed charge.	`includes/functions.php` – `calculate_shipping()`	How would you integrate a real shipping API later?
D2	What are the pros and cons of using **core PHP** instead of a framework?	Pros: low overhead, easy to host on free services, full control. Cons: more manual boiler-plate, higher risk of security bugs, harder to scale.	The project stays under ~12 k LOC, suitable for educational purposes and cheap hosting (InfinityFree). However, tasks like routing, validation, and ORM that are handled automatically in Laravel require explicit code here, increasing maintenance effort.	`documentation/learning-guide/04-PHP-and-Backend-Code.md`	Which framework would you pick for a rewrite and why?
D3	How does the code ensure **idempotency** of order creation?	The transaction + unique order number + row locks guarantee a single successful commit per request.	`place-order.php` generates an order number using `date('Ymd')` and a random hex; the unique constraint on `order_number` in the DB prevents duplicates. Even if the user refreshes, the PRG pattern (`header('Location: order-success.php')`) avoids re-submission.	`actions/place-order.php` lines 202-236	What would happen if the unique constraint were missing?
D4	Suggest one improvement to increase **test coverage** beyond the current 92 %.	Add integration tests for the admin dashboard's analytics queries.	The current suite covers core flows (login, checkout, cancellation). The admin dashboard aggregates data but isn't exercised; a test that inserts mock orders and asserts the revenue card shows the correct sum would raise coverage and catch future query regressions.	`tests/test-phase8.php` – test list	How would you mock the database for such a test?
D5	If you were to **containerise** this app, which services would you define?	A PHP-Apache container, a MySQL container, and optionally a phpMyAdmin container.	Dockerfile for the web server (PHP 8.1, Apache, extensions), `docker-compose.yml` linking it to a `mysql:8` service with the same schema, and a volume for `uploads/`. This mirrors the deployment guide's optional Docker section.	`Dockerfile` in the optional section of `09-Deployment-Guide.md`	How would you handle environment variables securely in Docker?

How to Use This List in Your Viva

1. **Read the short answers** first – they are your opening lines.
2. **Expand with the detailed answer** when the examiner asks "Explain…". Cite the file path or line numbers (e.g., "see `includes/csrf.php` line 23").
3. **Anticipate the follow-up** – prepare a one-sentence reply for the probe column.
4. **Practice aloud** – simulate an examiner and answer each question in under 30 seconds.
5. **Mark any gaps** – if you can't answer a probe confidently, revisit the associated file or the learning-guide sections.

Good luck!

Prepared by the senior core-PHP developer (AI-assisted).